ZAKIVAN

ALGERIA

Project Documentation

Caravan rental marketplace & interactive nature map of Algeria

Field	Value
Project	Zakivan Algeria
Version	1.0.0
Firebase project	zakivan-79cce
Cloudinary cloud	dvljsghr0
Framework	Next.js 14.2.35 (App Router)
Firestore collections	7
Composite indexes	9
Generated	07/10/2026

Table of Contents

1. Executive Summary
2. Feature Set
3. Technology Stack
4. System Architecture
5. Project Structure
6. Data Model — Firestore
7. Security Rules
8. Authentication & Roles
9. Electronic Contract System
10. Gift Card System
11. Cloudinary Image Pipeline
12. Firestore Indexes
13. Map Features
14. Environment Variables
15. Setup & Deployment
16. User Journeys
17. Operations Runbook
18. Production Roadmap
19. Legal & Compliance Notes

Appendix A — launch checklist

1. Executive Summary

Zakivan Algeria is a two-sided marketplace connecting private caravan owners with travellers across Algeria, paired with an interactive nature map of the country’s national parks, forests, and camping spots.

The platform deliberately operates **without escrow or any online payment processing**. Money moves directly between owner and renter, offline,

at vehicle handover. Zakivan’s role is limited to discovery, booking workflow, and evidentiary documentation. This keeps the business inside a regulatory envelope that avoids holding customer funds, while the electronic contract system supplies the legal record that offline settlement normally lacks.

Design pillars

- **No fund custody.** The platform never holds, collects, or transfers money on behalf

of anyone.

- **Evidentiary contracts.** Every booking produces a tamper-sealed acceptance record with identity, timestamp, IP address, and a SHA-256 content hash.
- **Client-first data.** Firestore is read and written directly from the browser SDK under strict security rules; no privileged backend sits in the request path.

  Secret-safe uploads. The Cloudinary API secret never leaves the server; the

browser
only
ever
receives a
short-lived signature.

At a glance

Metric	Value
Routes	13 (9 pages, 2 API routes, 1 asset)
Firestore collections	7
Composite indexes	9
User roles	3 (renter, owner, admin)
Booking states	5
Gift card instruments	2 (wallet credit, trip voucher)
Curated map locations	16
Shared JS payload	~88 kB First Load

2. Feature Set

2.1 Authentication

- Email/password registration and sign-in.
- Google OAuth sign-in with forced account selection.
- Three-role model on the user profile: `renter, owner, admin.`
- Self-service upgrade: a renter becomes an owner from the dashboard, which immediately unlocks listing creation and request management.
- Session persistence across reloads; route guards via RequireAuth.

2.2 Marketplace

Listing creation and editing

with rich specifications: capacity, beds, water tank litres, solar panel wattage, bathroom, kitchen, and air conditioning.

- Declarative extras list of equipment the owner includes.
- Publish / unpublish toggle.
- Filtering by wilaya, price band, guest capacity, caravan type, free text, and date availability.
- Owner-managed blocked availability ranges layered on top of real bookings.

2.3 Availability engine

A

date range is unavailable if it overlaps either a booking in an active state **(pending or accepted)** or an owner-blocked range. Availability is computed from the union of both sources before a request is allowed through.

2.4 Electronic contract system

Described in detail in section 9.

2.5 Booking lifecycle

State	Meaning	Who transitions
`pending`	Request submitted, awaiting owner review	— (initial)

State	Meaning	Who transitions
`accepted`	Owner confirmed the dates	Owner
`rejected`	Owner declined	Owner
`cancelled`	Renter withdrew the trip	Renter
`completed`	Trip finished and handed back	Owner

2.6 Gift cards

Described in detail in section 10.

2.7 Interactive nature map

- Leaflet map centred on Algeria with custom inline-SVG markers.
- Three location types: national park, forest, campsite.

  Filter chips with live per-type counts, plus free-text search over name and

wilaya.

- Detail drawer showing terrain, water availability, safety notes, practical tips, and best season.
- Responsive: slide-over list panel on mobile, fixed sidebar on desktop.
- Bounded to Algeria with map-boundary viscosity so users cannot pan into the sea.

3. Technology Stack

Layer	Technology	Version	Purpose
Framework	Next.js (App Router)	14.2.35	SSR, routing, API routes
Language	TypeScript	5.x	Type safety, strict mode
Styling	Tailwind CSS	3.x	Utility-first styling
Database	Cloud Firestore	—	Document store
Auth	Firebase Authentication	10.x	Email/password, Google
Media	Cloudinary	—	Image hosting & transforms
Maps	Leaflet + React-Leaflet	1.9 / 4.x	Interactive map
Icons	Lucide React	—	Icon set
Class merge	clsx	2.x	Conditional class names
Deploy	Firebase CLI	latest	Rules & index deployment

Why this stack

- **App Router** keeps data reads client-side while retaining file-based routing and colocated

API routes, so the deployment stays a single Node process.

- **Tailwind** avoids a component-library dependency and keeps the visual language consistent through theme tokens (brand, teal).
- **OSM tiles** rather than a commercial tile API: no key required, no billing account, and trivially swappable later by changing one URL string.

4. System Architecture

4.1 Request topology

```
Browser
    %
    % % % %º    N e x t . j s   A p p   R o u t e r   p a g e s   ( R e a c t   c l i e n t   c o m p o n e n t
    %              % % % %º    F i r e b a s e   A u t h   S D K           ( s i g n - i n ,   s e s s i o n
    %              % % % %º    F i r e s t o r e   W e b   S D K           ( a l l   r e a d s   &   w r i t
    %              % % % %º    C l o u d i n a r y   u p l o a d   A P I    ( d i r e c t   b r o w s e r   !'
    %
    % % % %º    N e x t . j s   A P I   r o u t e s
                 % % %      P O S T   / a p i / c l o u d i n a r y / s i g n      c o m p u t e s   a   S H A
                 % % %      G E T     / a p i / i p                              r e s o l v e s   t h e   c
```

4.2 Trust boundaries

Boundary	Enforcement
Client !' Firestore	`firestore.rules;` every query and write is authorised server-side by the Firestore security engine. The client is treated as untrusted.
Client !' Cloudinary	Signed uploads. The API secret is only read inside `/api/cloudinary/sign` and never returned to the browser.

Boundary	Enforcement
Client ! itself	Gift card arithmetic is re-derived on the server. A booking's totals must be internally consistent, and any credit it claims must match a hold that the same transaction placed on the card, so a tampered client cannot post an amount it did not spend. Availability, by contrast, is only checked in the client — see section 7.4.

4.3 The server is

not a privilege tier

There is no Firebase Admin SDK in the runtime. Consequently the API layer cannot bypass Firestore rules — it only performs two narrow, non-privileged jobs (signing an upload, detecting an IP). This is a deliberate simplification, and it is also why the security rules in section 7 carry the

full weight of authorisation.

4.4 Atomicity strategy

Multi-document writes are all-or-nothing. A batch applies every write in it or none of them, which is all that booking creation needs. A transaction adds reads and re-verification, which the gift card operations require because the rules judge a card write against a ledger row that only exists once the

write has been staged, and a refund has to read both the booking and the card’s current hold before it knows what to credit.

- **Booking creation** — booking document + contract document in one batch.

  Gift card redemption — booking + contract + card debit + ledger row in one transaction, with the debit recorded as a hold. This

is what prevents one card funding two bookings, and what makes a later release exact.

- **Gift card issuance** — card document + issued ledger row in one transaction.
- **Gift card claim and void** — card document + ledger row in one transaction.

A write that the rules reject fails as a whole, so a card can never be debited

without
its
booking,
and
a
booking
can
never
exist
claiming
credit
that
was
not
reserved.

5. Project Structure

```
app/
  layout.tsx                  Root layout: fonts, metadata, navbar, footer
  page.tsx                    Home: hero, search, featured caravans, map preview
  globals.css                 Tailwind layers + leaflet/prose overrides
  providers.tsx               Auth provider wrapper
  icon.svg                    Favicon
  caravans/page.tsx           Browse page (Suspense-wrapped browser)
  caravans/[id]/page.tsx      Detail page + booking flow
  map/page.tsx                Full interactive tourism map
  login/page.tsx              Sign-in
  register/page.tsx           Registration
  dashboard/
    page.tsx                  Listings / requests / trips / wallet / gift admin
    caravans/new/page.tsx     Create listing
    caravans/[id]/edit/page.tsx
  api/
    cloudinary/sign/route.ts  Signed-upload endpoint
    ip/route.ts               Client IP detection for contracts

components/
  caravans/
    CaravanCard.tsx           Listing tile
    CaravanGallery.tsx        Image carousel
    CaravansBrowser.tsx       Filter state + result list
    ImageUploader.tsx         Cloudinary upload with progress
    BookingPanel.tsx          Dates, pricing, gift card entry
    GiftCardRedeemer.tsx      Gift card code entry + live quote
  contract/
    ElectronicContractModal.tsx  Liability agreement + signature flow
  map/
    MapClient.tsx             ssr:false wrapper around the Leaflet map
    AlgeriaMap.tsx            Map container, tiles, markers, fit-bounds
    LocationDrawer.tsx        Selected location detail panel
    types.ts                  Icon registry for markers
  ui/
    Button.tsx, Modal.tsx, form.tsx
  layout/
    Navbar.tsx, Footer.tsx

lib/
  firebase.ts                 SDK init, lazy auth/db getters, collection names
  types.ts                    All domain interfaces and unions
  constants.ts                Wilayas, feature lists, status/type metadata
  utils.ts                    Dates, currency, overlap checks, SHA-256
  contract.ts                 Contract template builder + version constant
  cloudinary.ts               Signature fetch + XHR upload helper
  auth-context.tsx            AuthProvider, useAuth, RequireAuth
  services/
    users.ts, caravans.ts, bookings.ts, contracts.ts,
    locations.ts, giftcards.ts
  data/map-seed.ts            16 curated Algerian locations

scripts/
  generate-docs-pdf.mts       This document’s generator

firestore.rules               Role-based security rules
firestore.indexes.json        9 composite indexes
firebase.json                 Firebase CLI deploy config
```

```
.firebaserc                    Default project binding
```

6. Data Model — Firestore

6.1 users

Document ID is the Firebase Auth UID.

Field	Type	Notes
`uid`	string	Mirrors the document ID
`email`	string	
`displayName`	string	Falls back to email local-part
`photoURL`	string	Optional
`phone`	string	Used by owners at handover
`role`	renter | owner | admin	Drives all authorisations
`createdAt`	serverTimestamp	

6.2 caravans

Field	Type	Notes
`ownerId`	string	Auth UID; immutable after creation
`ownerName, ownerPhone`	string	Denormalised for handover

Field	Type	Notes
title, description	string	
type	caravan | motorhome | teardrop	
wilaya, commune	string	Algerian administrative divisions
locationGeo	{lat, lng}	Optional
pricePerDay	number	Algerian dinar
specs	object	capacity, beds, waterTankL, solarPanelW, hasBathroom, hasKitchen, hasAC
extras	string[]	Declared equipment
images	{url, publicId}[]	Cloudinary assets
unavailableRanges	{start, end}[]	Owner-blocked periods, epoch ms
published	boolean	Only published listings are publicly readable
createdAt, updatedAt	number	Epoch ms

6.3 bookings

Field	Type	Notes
caravanId, caravanTitle	string	Title denormalised so history survives deletion
ownerId, ownerName	string	
renterId, renterName	string	

Field	Type	Notes
`startDate, endDate`	number	Epoch ms
`days`	number	Derived, stored
`pricePerDay, totalPrice`	number	Price frozen at time of booking
`status`	BookingStatus	See section 2.5
`message`	string	Optional note to owner
`contractId`	string	FK to contracts
`giftCardId, giftCardCode`	string	Optional; the card code
`giftCardApplied`	number	Amount covered by the card
`cashDue`	number	totalPrice " giftCardApplied
`createdAt`	number	

> **Note: totalPrice** always reflects the full rental price shown to the owner. `cashDue` is what the renter physically hands over. A gift card never reduces what an owner is owed; it reduces what the renter pays.

6.4 contracts

The evidentiary record of one booking acceptance.

Field	Type	Notes
`bookingId, caravanId`	string	
`ownerId`	string	
`userId, userName, userEmail`	string	Signing party

Field	Type	Notes
`fullNameSigned`	string	Typed by the renter
`ipAddress`	string	From /api/ip
`userAgent`	string	Browser / device fingerprint
`contractVersion`	string	Template version used
`contentHash`	string	SHA-256 over version + terms + signature + uid + timestamp
`liabilityAcknowledged`	boolean	
`acceptedAt`	number	Epoch ms

> **Contracts are append-only** — rules permit create and read only, never update or delete.

6.5 giftCards

The document ID is the code itself (ZK-XXXX-XXXX-XXXX), so validation is a single document read

with no collection scan.

Field	Type	Notes
`code`	string	Duplicated inside the doc so ledger rows and rules can cross-check it
`kind`	wallet | voucher	Instrument type
`status`	active | used | void	
`initialAmount`	number	Face value, immutable
`balance`	number	Remaining credit
`heldBookingId`	string	The booking this card is currently reserved for; null when free
`heldAmount`	number	How much of the balance that reservation covers
`issuedBy`, `issuedByName`	string	Admin UID + display name

Field	Type	Notes
recipientUid	string	Set on claim; null while unclaimed
recipientEmail	string	Optional lock to an email address
recipientClaimedAt	number	
note	string	Free-text gift message
caravanId, caravanTitle	string	Vouchers only
startDate, endDate	number	Voucher validity window
expiresAt	number	Optional, both kinds
createdAt	number	
lastUsedAt, lastUsedBy	number, string	Last debit or refund audit trail
voidedAt, voidedBy	number, string	Set by the void operation

6.6 giftCardLedger

Append-only activity trail. Every row is written in the same transaction as the card

change it describes, and every row has a deterministic **document ID** so the operation can be replayed safely and verified by the rules:

Document ID	Operation
`issued_{code}`	Admin mints a card
`claimed_{code}`	Recipient attaches the card to their account
`applied_{bookingId}`	Credit reserved for a booking
`released_{bookingId}`	Reservation returned after cancel or reject
`voided_{code}`	Admin voids the card

Every row is written with every field present, using

an empty string where a field does not apply.

Field	Type	Notes
`giftCardId, code`	string	Must match the card document ID
`kind`	issued | claimed | applied | released | voided	
`amount`	number	Signed. Negative = debit
`balanceAfter`	number	Running balance snapshot
`bookingId`	string	For applied / released
`note`	string	
`actorUid`	string	Who caused it
`createdAt`	number	

6.7 mapLocations

Document ID is a stable slug from the seed data,

which makes re-seeding idempotent.

Field	Type	Notes
`name`, `type`, `wilaya`	string	type: national_park | forest | campsite
`geo`	{lat, lng}	
`terrain`, `water`	string	water: potable | seasonal | non_potable | none
`safetyNotes`	string	
`tips`	string[]	Practical traveller advice
`bestSeason`	string	
`description`	string	

6.8 Entity relationships

```
users (1) %% < (n) caravans
users (1) %% < (n) bookings as renterId
users (1) %% < (n) bookings as ownerId
users (1) %% < (n) contracts as userId
caravans (1) %% < (n) bookings
bookings (1) %% (1) contracts
bookings (0..1) %% > giftCards via giftCardCode
giftCards (1) %% < (n) giftCardLedger
users (1) %% < (n) giftCards via recipientUid
```

7. Security Rules

Rules are the only authorisation layer in this application. They are enforced by the Firestore backend on every request, so a compromised or modified client gains nothing.

7.1 Helper functions

Function	Returns
`isSignedIn()`	`request.auth != null`
`userDoc()`	The caller's users/{uid} document
`isAdmin()`	Signed in and role == `'admin'`

7.2 Per-collection policy

Collection	Read	Write

Collection	Read	Write
`users`	Self or admin	Create as renter/owner; self-edit cosmetic fields plus the single renter !’ owner upgrade; admin role is admin-only; delete admin
`caravans`	Published, or owner, or admin	Create if ownerId == uid; update/delete if owner or admin
`bookings`	Renter, owner, or admin	Create if renterId == uid and the totals are arithmetically consistent; owner may change only status, and only to accepted / rejected / completed; renter may only cancel

Collection	Read	Write
`contracts`	Signer, caravan owner, or admin	Create if userId == uid with an acknowledged liability and a non-empty hash; no update or delete
`mapLocations`	Public	Admin only
`giftCards`	Admin, recipient by UID, or email-locked holder	Create admin (with the issued row attached); claim by the locked email address; spend or release your own card under the hold rules; anything else admin

Collection	Read	Write
`giftCardLedger`	Admin or holder of the parent card	Append only. Each row must match the card write in the same transaction and use its deterministic document ID

7.3 The two rules that carry the most weight

Booking status transitions are field-restricted

An owner cannot rewrite history. The rule permits a single-field diff:

```
allow update: if isSignedIn() && (
  (
    resource.data.ownerId == request.auth.uid
    && request.resource.data.diff(resource.data)
        .affectedKeys().hasOnly(['status'])
  )
  || (
```

```
    resource.data.renterId == request.auth.uid
    && request.resource.data.status == 'cancelled'
    && resource.data.status in ['pending', 'accepted']
  )
);
```

The second clause means a renter can only ever set `cancelled`, and only from a state they are allowed to withdraw from. Changing `totalPrice or renterId` is impossible for either party, and the owner is further confined to the three statuses an owner actually owns.

A gift card moves in exactly one

of
three
shapes

Rather
than
one
permissive
"the
holder
may
edit
their
card"
rule,
each
of
the
three
legitimate
card
writes
is
its
own
rule,
and
each
one
has
to
be
paired
with
the
ledger
row that describes
it:

```
// claim — a card minted to this email, not yet attached
allow update: if isSignedIn()
  && resource.data.recipientUid == null
  && resource.data.recipientEmail == request.auth.token.email
  && request.resource.data.recipientUid == request.auth.uid
  && request.resource.data.balance == resource.data.balance
  && request.resource.data.diff(resource.data).affectedKeys()
      .hasOnly(['recipientUid', 'recipientName', 'recipientClaimedAt']);

// spend — balance only ever falls, and the debit is held
allow update: if isSignedIn()
  && resource.data.recipientUid == request.auth.uid
  && request.resource.data.balance <= resource.data.balance
  && request.resource.data.balance >= 0
  && request.resource.data.heldAmount
      == resource.data.balance - request.resource.data.balance
  && getAfter(ledgerPath('applied_' + request.resource.data.heldBookingId)).data.amount
      == -request.resource.data.heldAmount;
```

```
// release — only a cancelled or rejected booking, only its own hold, only once
allow update: if isSignedIn()
  && get(bookingPath(resource.data.heldBookingId)).data.status
      in ['cancelled', 'rejected']
  && request.resource.data.balance
      == resource.data.balance + resource.data.heldAmount
  && request.resource.data.heldBookingId == null
  && getAfter(ledgerPath('released_' + resource.data.heldBookingId)).data.amount
      == resource.data.heldAmount;
```

Three properties fall out of that shape. A holder can destroy their own credit but never mint any, because every branch that increases the balance is either admin-only or is pinned to an amount the card itself recorded. A refund can happen exactly once,

because it clears the hold. And no balance can move without a matching ledger row, because the row is verified against the card write in the same transaction via `getAfter`.

> **Why this is not simply "deny all ledger writes":** that variant, which this project used first, is unauditable in practice — the batch fails, so minting, claiming, and every redemption break, because a single batch is all-or-nothing. Ledger rows are therefore verified rather than forbidden. The rows are still immutable: updates and deletes are denied outright.

One card funds one booking at a time

A debit records heldBookingId and heldAmount on the card. A second booking would overwrite the hold, so the first reservation would become unreleasable. Rather than allowing an ambiguous state, the redemption service treats the hold as the card’s single in-flight reservation and the contract text instructs the renter to finish

or cancel the pending trip before booking the next one. This is a deliberate simplification, noted again in section 7.4.

7.4 Known limitations

- Availability conflict checks are advisory. Two owners could accept overlapping requests for the same caravan in the same instant. See section 18.
- A gift card reserves credit for one booking at

a time, so a second card-funded booking would overwrite the first hold. Supporting concurrent reservations needs a per-booking hold list, or a server-side re-validation function.

Ledger rows are verified against the card write they accompany, not against an independent source of truth. A trusted function that recomputes balances from the ledger remains the strongest

available audit.

- Admin role assignment has no self-service path, and this is enforced rather than merely absent: a profile can only be created as a renter or owner, and a self-update may only move role from renter to owner. The first admin must be written by hand in the console.

8. Authentication & Roles

8.1 Providers

Provider	Enabled	Setup location
Email / Password	Required	Console ’ Authentication ’ Sign-in method
Google	Required	Same screen; add the authorised domain

8.2 Role capabilities

Capability	renter	owner	admin
Browse published listings	yes	yes	yes
Request a booking	yes	yes	yes
View own trips	yes	yes	yes
Use gift cards / wallet	yes	yes	yes
Create & publish listings	—	yes	yes
Accept / reject requests	—	yes	yes
Block availability dates	—	yes	yes

Capability	renter	owner	admin
Mint / void gift cards	—	—	yes
Seed map locations	—	—	yes

8.3 Dashboard tabs by role

Role	Tabs
`renter`	My trips · Wallet & gift cards
`owner`	My listings · Booking requests · My trips · Wallet & gift cards
`admin`	Gift card admin · My listings · Booking requests · My trips · Wallet & gift cards

8.4 Account lifecycle

User registers or signs

in
with
Google.

2. `ensureUserProfile writes a users/{uid}`
document on
first sight,
defaulting role to renter.
3. If
a
Google
profile
arrives
with
a
name
it
is
written
once;
an
existing
name
is
never overwritten.
4. The renter
may self-upgrade
to owner
.
That
is
the
only
role
change
the
rules
permit
a
user
to
make
for
themselves;
`admin is`
not obtainable
from the
client.

9. Electronic Contract System

Version constant: ZAK-DZ-LIABILITY-v1.1

9.1 Purpose

Because settlement happens offline, the platform’s legal exposure is mitigated by producing a durable, tamper-evident record of what both parties agreed to. The template is explicitly a facilitation document: Zakivan is an intermediary, never a party to the lease and never a

custodian of funds.

9.2 The eleven articles

#	Article	Substance
1	Parties and Purpose	Identifies owner and tenant; states the intermediary role and no-fund-custody posture
2	Object of the Lease	Names the vehicle, type, declared capacity and extras; tenant accepts condition as described
3	Rental Period, Pickup and Return	Dates, handover location, same-condition return, per-day late compensation
4	Price and Payment	Total price, direct settlement, no escrow. Sub-clause records any gift card applied

#	Article	Substance
5	Tenant Liability	Full legal and financial liability for damage, loss, theft, fire, fines and towing costs
6	Insurance, Deposit, Handover	Deposit and insurance are private arrangements; absence does not reduce Article 5
7	Cancellation and Early Return	Free cancellation pre-acceptance; gift card credit returned on cancel or reject
8	Role of the Platform	Limited to hosting, search and contract facilitation; no warranty, no liability
9	Governing Law	Algerian law, Civil Code lease provisions, courts of the owner’s wilaya

#	Article	Substance
10	Electronic Signature	Law No. 15-04 equivalence, evidentiary value, hash sealing
11	Location Context	Destination wilaya hazards; tenant assesses route feasibility

9.3 Signature mechanics

1. The renter must tick an explicit acknowledgement of all clauses, with Article 5 called out by name.
2. The renter types their full legal name into a signature field (minimum 3 characters).

   `/api/ip` supplies the public IP,

shown live in the evidence panel.

4. A SHA-256 hash is computed over `contractVersion` | `terms` | `signature` | `uid` | `timestamp`.
5. Booking and contract are written in one all-or-nothing batch.

9.4 Why a hash rather than an immutable document

The hash seals the content. Any later alteration of the rendered terms would produce a

different digest, so the stored value is evidence that the exact text shown at signing is the text recorded. Firestore itself provides no document immutability and the rules block updates entirely, so the two controls are complementary: rules prevent casual tampering, and the hash makes tampering *detectable.*

9.5 Legal basis

Algerian Law No. 15-04 on electronic signature and

certification establishes that a qualified electronic document carries the same evidentiary value as a paper instrument. The template cites it directly in Article 10. Note that the app’s signature mechanism is a typed-name acknowledgement with a sealed hash, not a qualified certificate-based signature under the full PKI provisions of the law; that distinction should

be
confirmed
by
counsel
before
commercial
launch.

10. Gift Card System

10.1 The central constraint

Settlement is off-platform. Therefore a gift card can only ever be a **internal accounting instrument** that reduces what a renter owes at handover. It never moves money, never creates a receivable from an owner, and is never cash-redeemable. This keeps the feature entirely outside

payment regulation.

10.2 The two instruments

	Wallet credit	Trip voucher
Scope	Any listing on the platform	One specific caravan
Dates	Any available dates	Must fall inside the voucher window
Partial spend	Yes, remainder carries forward	Yes, remainder carries forward
Best for	Flexible gifts, referral credits, campaigns	Gifting a specific booked trip

10.3 Code format

Pattern ZK-XXXX-XXXX-XXXX. Twelve characters drawn from a 32-symbol alphabet that omits `0 O 1 I L` to remove transcription ambiguity, generated with `crypto.getRandomValues`. The code is also the document ID,

so
validation
is
a
single
`getDoc`.

10.4 Redemption flow

```
1. Renter picks dates on a listing
2. "Have a gift card?" expands ’ code entered
3. quoteGiftCard() validates:
     · exists · not void · not used · not expired · balance > 0
     · recipient UID is the signed-in account (so the card must already be
       in the wallet — an email-locked card has to be claimed first)
     · if voucher: caravanId matches AND dates inside window
   and returns { applied, cashDue, hint }
4. Booking panel recomputes: total, "applied, = due at
   (changing the caravan, dates or price discards the quote)
5. Contract re-renders with the gift card clause in Articles 4 and 7
6. On signing, the quote is re-validated against the live card and the
   contract is hashed over the freshly validated figures
7. One Firestore transaction writes:
     booking      (giftCardApplied, cashDue)
     contract
          giftCards        (balance "applied, status ’ used if f
                heldBookingId, heldAmount = applied)
     ledger       (applied_{bookingId}, negative amount)
8. If the owner rejects, or the renter cancels:
     releaseGiftCardForBooking() returns exactly the held amount and
     clears the hold, so a replay cannot credit twice
```

> **One booking per card at a time.** The hold occupies the card until the booking is accepted and completed, cancelled, or rejected. A second gift-card booking with the same card would overwrite that hold, so the booking panel is expected to steer the renter to finish the first trip first.

Claiming is a separate, earlier step, and

the distinction matters when reading the code. Adding a card to the wallet attaches it to an account; it does not spend it. The dashboard wallet holds the only claim interface, and the spend rule is written in terms of recipientUid rather than the email address, so a card that has been emailed but never claimed

cannot be used at checkout. When validation hits that case it says so and points the renter at the wallet instead of failing the transaction.

10.5 Validation matrix

Condition	Result
Unknown code	Rejected
Voided	Rejected
Fully used	Rejected
Past expiresAt	Rejected
Zero balance	Rejected
Claimed by a different UID	Rejected
Email-locked to a different address	Rejected

Condition	Result
Email-locked and not yet claimed	Rejected at checkout with a prompt to add it to the wallet
Voucher on the wrong caravan	Rejected
Voucher dates outside the window	Rejected
Balance below the trip price	Accepted; partial application, renter pays the remainder

10.6 Why the ledger is append-only and verified

A mutable balance field alone cannot answer “why does this account show zero?”. Every movement is therefore

also written as a signed ledger entry carrying `balanceAfter`, so the balance can be reconstructed and audited. Two rules keep the trail honest. It is immutable: updates and deletes are denied outright. And each row is verified against the card write it accompanies, so the row ID, its amount and its balance snapshot all

have to agree with the card as it will be after the transaction — a row cannot be invented to paper over a balance change.

10.7 Financial semantics

Situation	Card balance	Owner receives
Booking pending	Debited and held	Nothing yet
Owner rejects	Hold released in full	Nothing
Renter cancels	Hold released in full	Whatever the parties agree offline
Owner accepts	Stays debited	Full totalPrice, settled offline
Trip completed	Stays debited	Full totalPrice
Card voided by admin	Zeroed; any hold is forfeited	Unaffected

> **Important: the** owner is never a party to the gift card. Their payout is `totalPrice, never cashDue`. If a renter wins credit and then fails to pay the owner offline, that is a private debt between the two — Zakivan has no obligation and no funds. This is stated explicitly in Article 4 of the contract.

10.8 Expiry model

Expiry is evaluated at read time rather than by a scheduled job. There is no background worker to deploy or monitor, and no cron billing. A card past `expiresAt` fails validation, which is behaviourally identical to being voided, and its balance is simply unreachable. If you later want expiry emails or

automatic refunds, that becomes a Cloud Functions requirement.

10.9 Administrative controls

- **Mint** — kind, amount, recipient email (required), optional note, optional expiry; a voucher additionally requires a caravan and date window.
- **Void** — irreversible, zeroes the balance, writes a voided ledger entry. There is no un-void.

  List — every card with remaining vs.

face value, recipient, claimed state, and status.

Only role: 'admin' may mint or void, and the rules make that binding rather than conventional: a profile can only be created as a renter or owner, and a self-service update can only move role from renter to owner. To bootstrap the first admin, set `users/{uid}.role = 'admin'` by hand in the Firestore

console.

11. Cloudinary Image Pipeline

11.1 Why signed uploads

Cloudinary offers unsigned uploads, which require only a public API key and an upload preset — meaning anyone could upload to the account. This project instead signs uploads server-side so the API secret never reaches the browser.

11.2 The signing handshake

```
Browser                          Next.js                    Cloudinary
    %     P O S T   / a p i / c l o u d i n a r y / s i g n        %
    % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %°    %
    %                                                                    %   b u i l d   " f o l d e r = … & t i
    %                                                                    %   s h a 1 (   s t r i n g   +   A P I
    %     {   s i g n a t u r e ,   t i m e s t a m p ,                  %
    %         f o l d e r ,   a p i K e y ,   c l o u d N a m e   }      %
    % %Ä% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %$
    %
    %     P O S T   / a p i / v 1 _ 1 / { c l o u d } / i m a g e / u p l o a d    ( m u l t i p a r t ,   s i
    % % % % % % % % % % % % % % % % % % %
```

write-only storage — without it, a signed request could target any folder in the product environment.

11.4 Delivery transformations

Thumbnails are generated by URL rewriting, not by storing extra derivatives:

```
https://res.cloudinary.com/{cloud}/image/upload/v1/{publicId}.jpg       original
                !“   r e w r i t e
https://res.cloudinary.com/{cloud}/image/upload/w_800,q_auto,f_auto/…    thumb
```

- `w_800` — bounded width for cards and lists.
- `q_auto` — automatic quality selection; smaller files with no visible loss.
- `f_auto` —

automatic format negotiation (WebP / AVIF where supported).

Both q_auto and f_auto are available on the free plan. Because derivatives are cached at the CDN, a repeated request for the same URL is not re-derived and not re-billed — but any change to the transformation string produces a new billable derived asset, so keep the parameter set stable.

11.5 Storage model

Each listing stores an array of `{ url, publicId }`. The `publicId` is retained specifically so images can be destroyed later; the `url` alone would not allow it. Note that removing a photo from a listing currently only unlinks it from the array — see section 18 for the orphan-cleanup recommendation.

12. Firestore Indexes

Firestore indexes single fields automatically, but a query combining an equality filter with an ordering or a second filter requires a composite index. Nine are defined in firestore.indexes.json and deployed to the live project.

12.1 bookings

#	Fields	Serves query
1	`caravanId ASC, startDate ASC`	Booking timeline for one caravan
2	`ownerId ASC, createdAt DESC`	Owner’s incoming requests
3	`renterId ASC, createdAt DESC`	Renter’s trip history
4	`caravanId ASC, status ASC, endDate ASC`	Batch availability conflict sweep

12.2 caravans

#	Fields	Serves query
5	`published ASC, createdAt DESC`	Public browse feed
6	`ownerId ASC, createdAt DESC`	Owner's own listings, unpublished included
7	`ownerId ASC, published ASC, createdAt DESC`	Owner's published listings

12.3 giftCards and giftCardLedger

#	Fields	Serves query
8	`recipientUid ASC, createdAt DESC`	A user's cards
9	`giftCardId ASC, createdAt DESC`	A card's activity feed

Two queries deliberately need no composite index. The admin card list is an `orderBy` on a single field, which Firestore serves from its automatic index; and locating a

booking’s debit uses the deterministic ledger document ID `applied_{bookingId}` rather than a three-field filter, so it is a point read.

12.4 Queries that need no index

Equality-only queries are served by the automatic single-field indexes:

- `contracts where userId ==.`
- `contracts where bookingId ==.`
- `bookings where caravanId == and status in […].`
- `mapLocations unfiltered` collection read.

12.5 Deployment

```
firebase deploy --only firestore:indexes
```

When a query lacks an

index, Firestore also returns a console link in the browser’s error output; following it opens a pre-filled index creation form.

13. Map Features

13.1 Stack

React-Leaflet 4 with Leaflet 1.9, tiles served from the OpenStreetMap public endpoint:

```
https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png
```

No API key, no account, no billing. Attribution to OpenStreetMap contributors is declared and the attribution control is enabled.

13.2 SSR constraint

Leaflet touches window during module evaluation, which breaks server rendering. `MapClient` therefore loads the

map with `ssr: false`, and the page renders a spinner until the location query resolves. This is why the app's data reads are client-side rather than in Server Components.

13.3 Custom markers

Markers are inline SVG divIcons with a CSS class per type — not image sprites or icon fonts.

- Zero extra network requests

for marker assets.

- No CDN dependency for icon rendering.
- Colour and size are controlled by CSS classes, trivially restyleable.
- Emphasis for the selected location re-creates the icon instance, keyed on `id + selected` to force a re-render.

13.4 Data seeding

Sixteen curated Algerian locations ship as TypeScript in `lib/data/map-seed.ts`. On the first read of an empty mapLocations collection the client attempts

to
seed
it.
That
write
requires
the
admin
role,
so
for
non-admins
the
seed
silently
no-ops
and
the
page
falls
back
to
rendering
the
bundled
data
directly
—
the
map
is
fully
functional
before
Firestore
is
ever
populated.

> This fallback is deliberate. Without it, a permission error during seeding would abort the page and show an error instead of the map.

13.5 Tile usage policy

The public OSM tile servers are acceptable for development and low traffic, but are not intended for heavy production use. If traffic grows, replace the `url` prop

with MapTiler, Mapbox, Stadia, or a self-hosted tile server; only that one string changes.

13.6 Interaction details

Behaviour	Implementation
Initial framing	Auto-fit bounds over Algeria, padded
Pan containment	maxBounds with 0.7 viscosity, extended a few degrees
Deep-link to selection	Fly-to animation on selection change
Interactive vs preview	Zoom, drag, scroll, double-click, touch and keyboard toggled by a prop so the home page preview can disable them

14. Environment Variables

Held in .env.local, which is git-ignored. Two prefixes matter: NEXT_PUBLIC_ values are inlined into the browser bundle, everything else stays server-side.

14.1 Firebase (public)

Variable	Visibility	Notes
`NEXT_PUBLIC_FIREBASE_API_KEY`	Client	Identifies the project. Not a secret — access is governed by rules
`NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN`	Client	`zakivan-79cce.firebaseapp.com`
`NEXT_PUBLIC_FIREBASE_PROJECT_ID`	Client	`zakivan-79cce`
`NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET`	Client	Present for completeness; media uses Cloudinary
`NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID`	Client	
`NEXT_PUBLIC_FIREBASE_APP_ID`	Client	

14.2 Cloudinary

Variable	Visibility	Notes
`NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME`	Client	`dvljsghr0;` appears in delivery URLs, not sensitive
`NEXT_PUBLIC_CLOUDINARY_API_KEY`	Client	Public key; only useful with a valid signature
`CLOUDINARY_API_SECRET`	**Server only**	Signs uploads. Never exposed. Rotate if it ever leaks
`NEXT_PUBLIC_CLOUDINARY_FOLDER`	Client	Default zakivan/caravans ; must match the server's allowed prefix

14.3 Secret handling

- `.env.local and .env*.local are git-ignored.`
- The Cloudinary secret is read only inside `POST /api/cloudinary/sign.`

  No Firebase Admin SDK and no service account key

exists in the repository.

- The Firebase web API key is a public identifier by design; Firestore access is authorised by rules, not by the key.

15. Setup & Deployment

15.1 Local development

```
npm install
cp .env.example .env.local     # then fill in the values
npm run dev                    # http://localhost:3000
```

Other scripts: npm run `build` for a production build, npm `run start` to serve it, npx tsc `--noEmit` for a type check.

15.2 Firebase project configuration

1. Web app created in project zakivan-79cce; config copied into env vars.
2. Authentication providers enabled: Email/Password and Google.
3. Firestore created in production mode.
4. Composite indexes deployed.
5. Security rules deployed.

15.3 Deploy commands

```
firebase deploy --only firestore:rules
firebase deploy --only firestore:indexes
firebase deploy --only firestore:rules,firestore:indexes   # both
```

Bound to the project via .firebaserc; deploy targets declared in `firebase.json`.

15.4 Bootstrapping the first admin

1. Register normally through the app.
2. Copy the UID from the Firebase console, or read it from the browser console via `getAuth(getApp()).currentUser.uid`.
3. In Firestore, open `users/{uid}` and set role to "admin".
4. Reload the dashboard; the **Gift card admin tab** appears.

> There is intentionally no UI path to assign this role, and the rules enforce that: a self-service profile update is limited to cosmetic fields plus the renter !’ owner step, so role: admin can only be written by an admin.

15.5 Cloudinary configuration

1. Note the cloud name on the Settings !’ Account page.

   Copy the API key and

API secret from Settings !’ API Keys.

3. Place all three into .env.local.

No unsigned upload preset is required, because uploads are signed. The `zakivan/caravans folder` is created automatically on first upload.

15.6 Manual map seeding

Open /map while signed in as an admin. The client seeds mapLocations on first read of an empty collection. Afterwards the collection is served from Firestore.

16. User Journeys

16.1 Renter: browse to booked

```
Home !’ search widget (wilaya, dates, guests, type)
       !’ /caravans results, filtered client-side
       !’ listing detail: gallery, specs, extras, map conte
       !’ Booking panel: pick dates !’ live price
       !’ (optional) apply gift card !’ see applied + cash d
       !’ sign contract (tick + type name + IP shown)
       !’ booking created pending; ledger debited if a card
       !’ Dashboard !’ My trips !’ track status
       !’ owner accepts !’ renter may cancel until completio
```

16.2 Renter: redeem a gifted card

```
Gift received (email or message) with code ZK-XXXX-XXXX-XXXX
   !’ Dashboard !’ Wallet & gift cards
   !’ paste code !’ Redeem
   !’ email must match the account
   !’ card attaches to the account (recipientUid set)
   !’ appears under "My cards" with remaining balance
   !’ future bookings can draw on it
```

16.3 Owner: list and manage

```
Register !’ Dashboard !’ "Become an owner"
   !’ Add caravan: photos upload straight to Cloudinary wi
   !’ set specs, extras, price, wilaya
   !’ publish when ready
   !’ Dashboard !’ My listings !’ edit / delete / manage ava
   !’ Booking requests: accept or reject
     · reject releases any gift card reservation
   !’ accepted !’ Mark completed after the trip
```

16.4 Admin: issue gift cards

```
Dashboard !’ Gift card admin
  !’ choose Wallet credit or Trip voucher
  !’ amount, recipient email (required), optional note, o
  !’ voucher: pick a listing + validity window
  !’ Create !’ code generated and shown
  !’ List shows all cards with remaining balance and stat
  !’ Void to invalidate (irreversible)
```

17. Operations Runbook

17.1 Routine checks

Cadence	Check	Where
Weekly	Pending request age	Dashboard !’ Booking requests
Weekly	Gift card liability outstanding	Dashboard !’ Gift card admin (sum of remaining balances)
Monthly	Cloudinary usage vs. plan	Cloudinary console !’ Account
Monthly	Firestore reads and stored bytes	Firebase console !’ Usage
Per release	Rules & index deploy	`firebase deploy --only firestore`

17.2 Financial exposure to monitor

The sum of all active balance values in giftCards is a contingent liability, not cash: it represents

future reductions of amounts renters owe owners. Since owners are paid the full price offline, a large outstanding balance is an operational risk only in the sense that renters may have “paid” with credit and then fail to settle with the owner privately. Monitor card-funded pending bookings closely.

17.3 Common issues

Symptom	Likely cause	Resolution

Symptom	Likely cause	Resolution
Map shows “Could not load map locations”	Seed write denied (not admin), or an unreadable collection	Non-admins now get bundled fallback data. If it persists, inspect the collection
Query fails with “requires an index”	Missing composite index	Follow the console link, or add to firestore.indexes.json and deploy
“Gift card admin” tab missing	User role is not admin	Set users/{uid}.role = `'admin'` in the console
A minted code is “not recognised”	The card was written under a random ID instead of its code	Void it and mint again; codes are now document IDs
Redemption is rejected with a permission error	The booking, card and ledger writes must land in one transaction	Keep the three writes together; a partial retry will not satisfy the rules

Symptom	Likely cause	Resolution
Refund does not appear after a cancel	No hold on the card, or the booking was never card-funded	Check heldBookingId on the card and released_{bookingId} in the ledger
“Cloudinary is not configured on the server”	Missing server env var	Check the three Cloudinary values in .env.local
Upload rejects the folder	Folder outside the allowed prefix	Keep the folder under `zakivan/`
Permission denied writing a booking	Contract create requires userId == uid	Sign in; unauthenticated requests cannot create
Contract tampering suspected	—	Recompute SHA-256 over version, terms, signature, uid, timestamp and compare with contentHash

17.4 Rotation procedure

If the Cloudinary API secret is ever exposed, rotate it in

the Cloudinary console and update CLOUDINARY_API_SECRET. Existing signatures expire naturally because they embed a timestamp, so the impact window is minutes. If the Firebase web API key leaks, no rotation is needed — it is public by design and rules are the real control.

18. Production Roadmap

18.1 High priority

Item	Rationale
Transactional availability re-check	Conflict detection today runs client-side before submission. Under concurrency two owners could accept overlapping dates. A Cloud Function should re-validate overlaps inside a transaction at booking creation.

Item	Rationale
Cloudinary orphan cleanup	Removing a photo only unlinks it from the listing, while the asset keeps costing storage and transformations. A scheduled function should reconcile listings against stored public IDs and destroy the difference.
Gift card abuse monitoring	Alert on cards with rapid issue-and-redeem cycles and on balances never used. Consider a per-account issuance cap.

Item	Rationale
Gift card balance re-derivation	The rules verify each ledger row against the card write it accompanies, which is strong but still client-driven. A Cloud Function recomputing balances from the ledger would make the ledger, not the client, the source of truth, and could also lift the one-booking-per-card-at-a-time restriction.

Item	Rationale
Contract PDF export	The contract is currently only rendered in-app. Emitting a signed PDF for renter and owner would materially strengthen the evidentiary position.

18.2 Medium priority

Item	Rationale
Server-side reads for SEO routes	Client-only fetching keeps listings out of search results. Migrating browse and detail to Server Components with force-dynamic would make them indexable.

Item	Rationale
Notifications	Email or push on request, acceptance and rejection. Today the owner must open the dashboard to discover new work.
Pagination	Browse and dashboard cap at 200 records. Beyond that, add cursor pagination.
Reviews and ratings	No trust signal exists on listings yet. Given the weight of Article 5, owner history matters.
Multi-currency display	Prices are DZD-only. A EUR reference rate would help diaspora renters.

18.3 Lower priority

- Wishlists and price alerts.
- Owner payout reporting and tax export.
- Seasonal demand analytics per wilaya.
- Gift card bulk import for corporate or campaign use.
- Offline map tiles for remote Saharan destinations, where connectivity is unreliable.

18.4 Deliberately excluded

- **Online payment capture.** Would introduce fund custody and the regulatory burden the current design deliberately avoids.

- **Gift card expiry automation**. Read-time evaluation is sufficient until there are enough cards to justify a scheduled job.
- **Escrow.** Same reasoning; contracts, not custody, are the risk control.

19. Legal & Compliance Notes

> This section records what the code does and the reasoning behind it. It is not legal advice. Have counsel review before commercial launch.

19.1 Data protection

Data	Collected	Basis / control
Contract acceptance	Full legal name, IP, user agent, timestamp	Contract performance; evidenced by explicit tick plus typed signature

Data	Collected	Basis / control
Profile	Email, display name, phone, photo	Account operation; phone is used for handover coordination
Listings	Vehicle description, photos, location	Published by the owner intentionally

At time of writing the application records no consent checkbox for data processing, sets no retention period, and offers no account deletion or data-export path. For Algerian users, Law No. 18-07 on personal data protection is the governing regime.

Three additions are advisable before launch: an explicit consent record, a retention policy for contract evidence, and a self-service account deletion path that anonymises rather than erases the contract record.

19.2 Contract enforceability

The template cites **Law No. 15-04** for the equivalence of electronic and paper instruments. Two caveats:

- The signature is a typed-name acknowledgement sealed with

a hash, not a qualified certificate signature under a trusted PKI. Whether that satisfies 15-04’s formal requirements in a given dispute is a question for counsel.

Article 5’s full-liability language and Article 9’s jurisdiction clause should be reviewed for fairness and enforceability under Algerian consumer law, particularly where the tenant is a consumer rather

than a business.

19.3 Gift cards as an instrument

Because gift cards never touch money and never reduce an owner's payout, they are best characterised as promotional credit rather than stored value. Two consequences:

- **No cash redemption, ever.** Article 7 and the checkout UI both state this. It is the main protection against the

instrument being treated as stored value.

- **Expiry.** Unrestricted perpetual credit would raise abandoned-balance and accounting questions. Expiry is available per card, though not enforced by a job.

19.4 Platform liability posture

Article 8 disclaims warranty as to vehicle condition and liability for bodily injury, material damage and financial loss. Combined with the

absence of escrow and fund custody, the platform is structured as an information intermediary. Note that such disclaimers are not absolute: under Algerian law liability cannot be excluded for intentional misconduct, and consumer protection rules may restrict exclusion of warranty for hidden defects. This posture is a reasonable starting point, not a shield.

19.5 Map data accuracy

Coordinates, terrain, water availability and safety notes in `lib/data/map-seed.ts` are curated starting points for a demo, not verified cartography. They are rendered in the UI as traveller guidance, including safety information. Before launch, each entry should be field-verified, dated and sourced, and a disclaimer that the information is indicative and unverified

should be visible in the drawer.

19.6 Cloudinary and content

Owners upload their own photography. Confirm that a reported or disputed image can be removed from the delivery CDN, and establish a takedown contact. Because assets are served from Cloudinary rather than the application origin, takedown must happen at the media provider.

Appendix A — Launch Checklist

A.1 Firebase

```
[ ] Email/Password sign-in provider enabled
[ ] Google sign-in provider enabled, authorised domain added
[ ] Firestore created in production mode
[ ] Security rules deployed:  firebase deploy --only firestore:rules
[ ] Composite indexes deployed: firebase deploy --only firestore:indexes
[ ] /map opened once as admin to seed mapLocations
[ ] First admin role assigned in the console
```

A.2 Cloudinary

```
[ ] Cloud name copied into NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME
[ ] API key copied into NEXT_PUBLIC_CLOUDINARY_API_KEY
[ ] API secret copied into CLOUDINARY_API_SECRET (server only)
[ ] Folder prefix kept under zakivan/
[ ] One test upload completed end to end
```

A.3 Application

```
[ ] .env.local present and git-ignored
[ ] npm run build succeeds
[ ] npx tsc --noEmit clean
[ ] Branding shows Zakivan everywhere
[ ] Test booking completed with and without a gift card
[ ] Test card rejection returns the balance
```